

Queues, Event Streams, and Reliable Processing

A queue decouples arrival from processing, but does not create infinite capacity or remove failure. An event log supports retained history and replay, but consumers still need correct state transitions. Define delivery, processing, and business effects separately before making reliability claims.

For: Engineers who want to explain asynchronous processing, ordering, acknowledgement, retries, backpressure, and replay beyond simply adding a message broker to a diagram.

Suggested pace: 3 weeks with a durable-worker exercise | Backend fundamentals

What you will be able to do

- Choose queue or log semantics according to consumption and replay needs.
- Place acknowledgements after the intended durable processing boundary.
- Handle duplicates, poison messages, ordering, and bounded retries.
- Estimate backlog and recovery time while enforcing fairness and backpressure.

Build the right mental model

Delivery is not completion

A broker can deliver a message while the worker crashes before or after applying its effect. Acknowledging too early loses work; acknowledging after a durable effect permits redelivery and therefore requires deduplication or idempotency.

Ordering has a scope

A partition or queue may preserve one order, but global ordering across independent partitions is a different and more expensive requirement. Key events according to the entity whose transitions need order, and account for hot-key throughput limits.

Backlogs are stored overload

If arrivals exceed processing capacity, the backlog grows. Retention, disk, and deadlines are finite. Rate limits, shedding, prioritization, and producer feedback are part of the design; a queue only buys time to recover.

Study with evidence.

Build a worker with a durable task ID and explicit states. Crash it at controlled points and replay work. Keep effects local or mocked so failure testing never sends real notifications, charges money, or modifies external systems.

Week 1: Contracts and state

Define accepted, queued, processing, completed, failed, and cancelled states. Choose message identity and payload version. Implement a producer and worker with acknowledgement after durable completion.

Deliverable and check: A lifecycle diagram and tests for crash-before-effect and crash-after-effect-before-acknowledgement.

Week 2, first half: Retries and duplicates

Add bounded exponential backoff with jitter, retryable versus terminal errors, and a dead-letter review process. Make local effects idempotent under a stable ID and reject conflicting reuse of that ID.

Deliverable and check: A duplicate-delivery test and a poison-message case that does not block the whole queue indefinitely.

Week 2, second half: Ordering and replay

Choose a partition key and explain per-key order. Test consumer restart, checkpoint recovery, and schema evolution. Distinguish reprocessing a log from applying a business effect twice.

Deliverable and check: A replay plan that preserves output correctness and records the processing version.

Week 3: Capacity and operations

Measure arrival rate, processing rate, queue age, retry rate, and lag. Calculate recovery time for a finite backlog. Add admission limits and tenant fairness so one producer cannot consume all capacity.

Deliverable and check: An overload runbook with bounded storage and explicit behavior when recovery cannot meet deadlines.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Calculate and control a growing backlog

A worker pool can process 800 jobs/second while producers submit 1,000 jobs/second for ten minutes. Each queued job occupies about 2 KB. Determine the accumulated backlog and how recovery depends on later traffic.

A worked approach

1. The excess rate is 200 jobs/second. Over 600 seconds, the backlog grows by 120,000 jobs, consuming about 240 MB of raw job data before broker overhead, replicas, and retained retry history.
2. If arrivals later fall to 500 jobs/second while processing remains 800, the net drain is 300 jobs/second. The idealized backlog drain time is $120,000/300 = 400$ seconds, excluding variability and failed work.
3. If arrivals remain 1,000, the backlog never drains at the same capacity. Add a capacity or admission response, and monitor oldest-job age rather than only queue length. Protect high-priority and other tenants' work with explicit budgets.

Expected result

The queue's role and limits become quantitative. Storage buys a bounded recovery window, but sustained excess demand requires a real change in capacity, workload, or admission policy.

Verify the result

- Are retries included in effective arrival load?
- Does retention exceed the required recovery window?
- Can a hot tenant or poison message starve unrelated work?

Extend the exercise

Crash a worker after applying a local database effect but before acknowledgement. Use a transactionally recorded job ID to avoid applying that effect again on redelivery, and name the limits of this guarantee for external APIs.

Make the learning stick.

Does a message broker guarantee exactly-once business effects?

Not by itself. Broker transactions may cover a defined boundary, while an external email, payment, or database write may be outside it. State the transaction, identity, and reconciliation boundaries explicitly.

When should a message enter a dead-letter queue?

After a defined terminal error or bounded retry policy, with enough sanitized context for investigation. A dead-letter queue needs an owner and replay policy; otherwise it is merely a place where failed work is forgotten.

Why monitor age as well as length?

A short queue can contain one very old stuck item, while a large queue of fast jobs may still meet deadlines. Age and lag connect backlog to user impact more directly than count alone.

Avoid these traps

- Acknowledging before the promised durable effect.
- Treating an unbounded queue as a scaling strategy.
- Replaying events without versioning or duplicate-effect handling.

Ready for the next step?

- Acknowledgement and effect boundaries are explicit.
- Retries and poison messages have bounded behavior.
- Ordering scope and partition keys are justified.
- Backlog growth and recovery time can be estimated.

Reliable next reads

- [Apache Kafka documentation](#)
- [Azure queue-based load leveling](#)
- [RabbitMQ reliability guidance](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.